

MaxHeap en Python

Un MaxHeap es una estructura de datos que mantiene el mayor elemento en la raíz. Aquí te muestro cómo implementarlo en Python, explicando cada parte.

Implementación

```
class MaxHeap:
    def __init__(self):
        self.heap = []

    def swapp(self, i, j):
        """Intercambia dos nodos en el heap."""
        self.heap[i], self.heap[j] = self.heap[j], self.heap[i]

    def burbujeo_arriba(self, index):
        """Realiza el burbujeo hacia arriba para mantener la propiedad del
        MaxHeap."""
        parent = (index - 1) // 2
        if index > 0 and self.heap[index] > self.heap[parent]:
            self.swapp(index, parent)
            self.burbujeo_arriba(parent)

    def burbujeo_abajo(self, index):
        """Realiza el burbujeo hacia abajo para mantener la propiedad del
        MaxHeap."""
        left = 2 * index + 1
        right = 2 * index + 2
        largest = index

        if left < len(self.heap) and self.heap[left] > self.heap[largest]:
            largest = left
        if right < len(self.heap) and self.heap[right] > self.heap[largest]:
            largest = right
        if largest != index:
            self.swapp(index, largest)
            self.burbujeo_abajo(largest)

    def insert(self, value):
        """Inserta un nuevo valor en el heap."""
        self.heap.append(value)
        self.burbujeo_arriba(len(self.heap) - 1)

    def extract_max(self):
        """Extrae y devuelve el máximo valor del heap."""
        if len(self.heap) == 0:
            return None
        if len(self.heap) == 1:
            return self.heap.pop()
```

```
root = self.heap[0]
self.heap[0] = self.heap.pop()
self.burbujeo_abajo(0)
return root
```

Uso

```
max_heap = MaxHeap()
max_heap.insert(10)
max_heap.insert(20)
max_heap.insert(5)
print(max_heap.extract_max()) # Salida: 20
print(max_heap.extract_max()) # Salida: 10
print(max_heap.extract_max()) # Salida: 5
```

Este código define una clase `MaxHeap` con métodos para insertar elementos y extraer el máximo, manteniendo la propiedad del heap mediante los métodos `burbujeo_arriba` y `burbujeo_abajo`.

```
def eliminar(self, value):
    """Elimina un valor específico del heap."""
    try:
        index = self.heap.index(value)
        # Mover el último elemento al lugar del elemento a eliminar
        self.heap[index] = self.heap.pop()
        # Restaurar la propiedad del heap
        if index < len(self.heap):
            self.burbujeo_abajo(index)
            self.burbujeo_arriba(index)
    except ValueError:
        print("El valor no está en el heap.")
```

Ejemplo de eliminación

```
max_heap = MaxHeap()
max_heap.insert(10)
max_heap.insert(20)
max_heap.insert(5)
max_heap.eliminar(20)
print(max_heap.extract_max()) # Salida: 10
print(max_heap.extract_max()) # Salida: 5
```

Este método `eliminar` permite eliminar un valor específico del heap. Si el valor no se encuentra en el heap, se imprime un mensaje de error. Después de eliminar el valor, se restaura la propiedad del heap

mediante los métodos `burbujeo_arriba` y `burbujeo_abajo`.

Ejemplo visual de MaxHeap

Lista

```
Insertar 10:  
[10]  
  
Insertar 20:  
[20, 10]  
  
Insertar 5:  
[20, 10, 5]  
  
Extraer máximo (20):  
[10, 5]  
  
Extraer máximo (10):  
[5]  
  
Extraer máximo (5):  
[]
```

Árbol Binario

```
Insertar 10:  
  10  
  
Insertar 20:  
  20  
 /  
10  
  
Insertar 5:  
  20  
 / \  
10  5  
  
Extraer máximo (20):  
  10  
 /  
  5  
  
Extraer máximo (10):  
  5  
  
Extraer máximo (5):  
  (vacío)
```

Ejemplo visual de eliminación

Lista

```
Insertar 10:  
[10]  
  
Insertar 20:  
[20, 10]  
  
Insertar 5:  
[20, 10, 5]  
  
Eliminar 20:  
[5, 10]  
  
Extraer máximo (10):  
[5]  
  
Extraer máximo (5):  
[]
```

Árbol Binario

```
Insertar 10:  
  10  
  
Insertar 20:  
  20  
 /  
10  
  
Insertar 5:  
  20  
 / \  
10  5  
  
Eliminar 20:  
  10  
 /  
 5  
  
Extraer máximo (10):  
  5  
  
Extraer máximo (5):  
  (vacío)
```

